

Version: 1.0.0 · **Last Updated:** 2026-06-01
Target Region: Kuta, Lombok, NTB, Indonesia
Payment Model: Cash on Delivery (COD) — MVP

Payment Model: Cash on Delivery (COD) — MVP

```

graph TD
    subgraph Frontend
        direction LR
        C[Customer App]
        D[Driver App]
        M[Merchant/ Resto App]
        A[Admin Panel (React)]
    end

    C --> B[HTTPS / JWT]
    D --> B
    M --> B
    A --> B

    B --> Backend[KUWRIR Backend API  
Go + Gin · :8080/api/v1]

    Backend --> PG[PG16 PostGIS]
    Backend --> R[Redis7 Cache]
    Backend --> V[Valhalla Routing]
    Backend --> N[Nominatim Geocoding]

    Backend --> CS[Cloudflare R2 Image Storage]
    Backend --> U["./uploads/ (local) placeholder now"]
    U --> SR[← swap-ready]
  
```

The diagram illustrates the KUWRIR Platform Architecture. At the top, four front-end applications are listed: Customer App, Driver App, Merchant/ Resto App, and Admin Panel (React). These applications connect to the KUWRIR Backend API (Go + Gin · :8080/api/v1) via HTTPS / JWT. The Backend API then interacts with four services: PG16 PostGIS, Redis7 Cache, Valhalla Routing, and Nominatim Geocoding. Additionally, the Backend API connects to Cloudflare R2 Image Storage and a local uploads directory (./uploads/ (local) placeholder now), which is noted as being swap-ready.

2. Technology Stack

Layer	Technology	Version
Backend API	Go + Gin Framework	Go 1.22+, Gin 1.10
ORM	GORM	v2
Database	PostgreSQL + PostGIS	16 + 3.4
Cache	Redis	7 Alpine
Routing Engine	Valhalla	Latest
Geocoding	Nominatim	Latest
Admin Panel	React + Vite + TypeScript	React 19, Vite 8
Admin UI Kit	shadcn/ui + Tailwind CSS v4	Latest
Mobile Apps	Flutter	3.x
Shared Package	Dart (kuwrir_shared)	0.1.0
Image Storage	Local disk <code>./uploads/</code> → Cloudflare R2 (swap-ready)	—
Auth	JWT (access + refresh tokens)	HS256 / bcrypt

3. Project Structure

```

kuwrir-aja/
├── backend/
│   ├── cmd/server/main.go
│   ├── internal/
│   │   ├── config/config.go
│   │   ├── handler/
│   │   │   ├── admin/handler.go          # Admin CRUD + driver apps + settlements +
promos
│   │   │   ├── auth/handler.go          # Register + Login (role-aware)
│   │   │   ├── customer/handler.go      # Food orders, driver orders
│   │   │   ├── driverreg/handler.go     # Driver application + document upload
│   │   │   └── kasir/handler.go         # POS/Kasir (transactions, stock, piutang,
hutang, reports)
│   │       ├── merchant/handler.go      # Merchant CRUD + self-delivery
│   │       ├── service/handler.go       # Service (jasa) orders - full 2-leg flow
│   │       ├── middleware/auth.go
│   │       ├── model/models.go         # 22+ GORM models
│   │       └── upload/upload.go         # File upload helper (local → R2 swap-ready)
│   ├── uploads/                        # Local document storage (placeholder)
│   │   ├── driver-docs/
│   │   └── merchant-docs/
│   └── docker-compose.yml
├── admin_panel/src/pages/
│   ├── DashboardPage.tsx              # Live KPI stats
│   ├── LoginPage.tsx
│   ├── MerchantsPage.tsx              # Approve/reject merchants
│   ├── DriversPage.tsx                # Suspend + COD deposit management
│   ├── DriverApplicationsPage.tsx     # Review driver registrations + doc preview
│   ├── CustomersPage.tsx              # Suspend/activate users
│   ├── OrdersPage.tsx
│   ├── SettlementsPage.tsx            # Per-merchant payout + Mark as Paid
│   ├── PromotionsPage.tsx             # Full promo CRUD
│   └── SettingsPage.tsx               # Configurable fees
├── customer_app/lib/screens/
│   ├── home_screen.dart               # Food merchant browse
│   ├── merchant_detail_screen.dart
│   ├── cart_screen.dart
│   ├── order_tracking_screen.dart
│   ├── search_screen.dart
│   ├── service_home_screen.dart       # Jasa merchant browse
│   ├── service_booking_screen.dart    # Book service + schedule pickup
│   └── service_tracking_screen.dart    # 8-step service timeline
├── merchant_app/lib/screens/
│   ├── login_screen.dart              # Merchant login (with register link)
│   ├── register_screen.dart           # 3-step merchant registration + doc upload
│   ├── pending_screen.dart            # Verification status polling
│   ├── orders_screen.dart             # Live food order queue
│   ├── menu_screen.dart
│   ├── store_screen.dart
│   ├── service_orders_screen.dart     # 4-tab service order management
│   ├── kasir_screen.dart              # POS Terminal
│   ├── kasir_reports_screen.dart      # Laba Rugi, Arus Kas, Stok
│   └── kasir_receivables_screen.dart  # Piutang/tab management

```

```
| └─ kasir_payables_screen.dart      # Hutang supplier
|
| └─ driver_app/lib/screens/
|   └─ login_screen.dart
|   └─ register_screen.dart          # Driver registration step 1
|   └─ onboarding_screen.dart        # Upload 5 documents + vehicle info
|   └─ pending_screen.dart           # Verification status polling
|   └─ job_board_screen.dart          # 2-tab: Food + Service jobs
|   └─ service_jobs_screen.dart       # Service pickup & return jobs
|   └─ active_delivery_screen.dart
|   └─ wallet_screen.dart
|
| └─ shared/kuwrir_shared/            # Shared Dart package
| └─ finansial-mac/                  # Reference accounting app (Python/Flask)
| └─ SPECIFICATION.md
| └─ CHANGELOG.md
| └─ SYSTEM_OVERVIEW.md              # Business proposal + full system design
| └─ project_proposal.md
```

4. Database Models (22+ tables)

4.1 Core Models

#	Model	Key Fields
1	User	name, email, phone, role (customer/driver/merchant/admin), is_active
2	Address	user_id, label, lat/lng, is_default
3	Merchant	user_id, name, type (food/service), service_category, is_verified, verification_status
4	ProductCategory	merchant_id, name, sort_order
5	Product	category_id, name, price, cost_price, price_unit, duration_estimate, track_stock, stock_quantity, min_stock
6	ProductVariant	product_id, group_name, name, price, is_required
7	Driver	user_id, vehicle_type, vehicle_plate, is_online, cash_balance
8	Order	order_number, service_type (ecommerce/service/pos), status, pickup_scheduled_at, service_notes, weight_kg
9	OrderItem	order_id, product_id, item_name, quantity, base_price, unit_price
10	Review	order_id, customer_id, merchant_rating, driver_rating
11	SystemSetting	key, value, label
12	DriverDeposit	driver_id, amount, method, verified_by_id
13	MerchantSettlement	merchant_id, period_start/end, total_orders, total_base_product_amount, status
14	Promotion	code, type (percentage/fixed/free_delivery), value, usage_limit, starts_at, expires_at

4.2 Registration & Verification Models

#	Model	Key Fields
15	DriverApplication	user_id, vehicle_type, vehicle_plate, ktp_url, sim_url, stnk_url, selfie_url, vehicle_photo_url, status (pending/approved/rejected), review_note

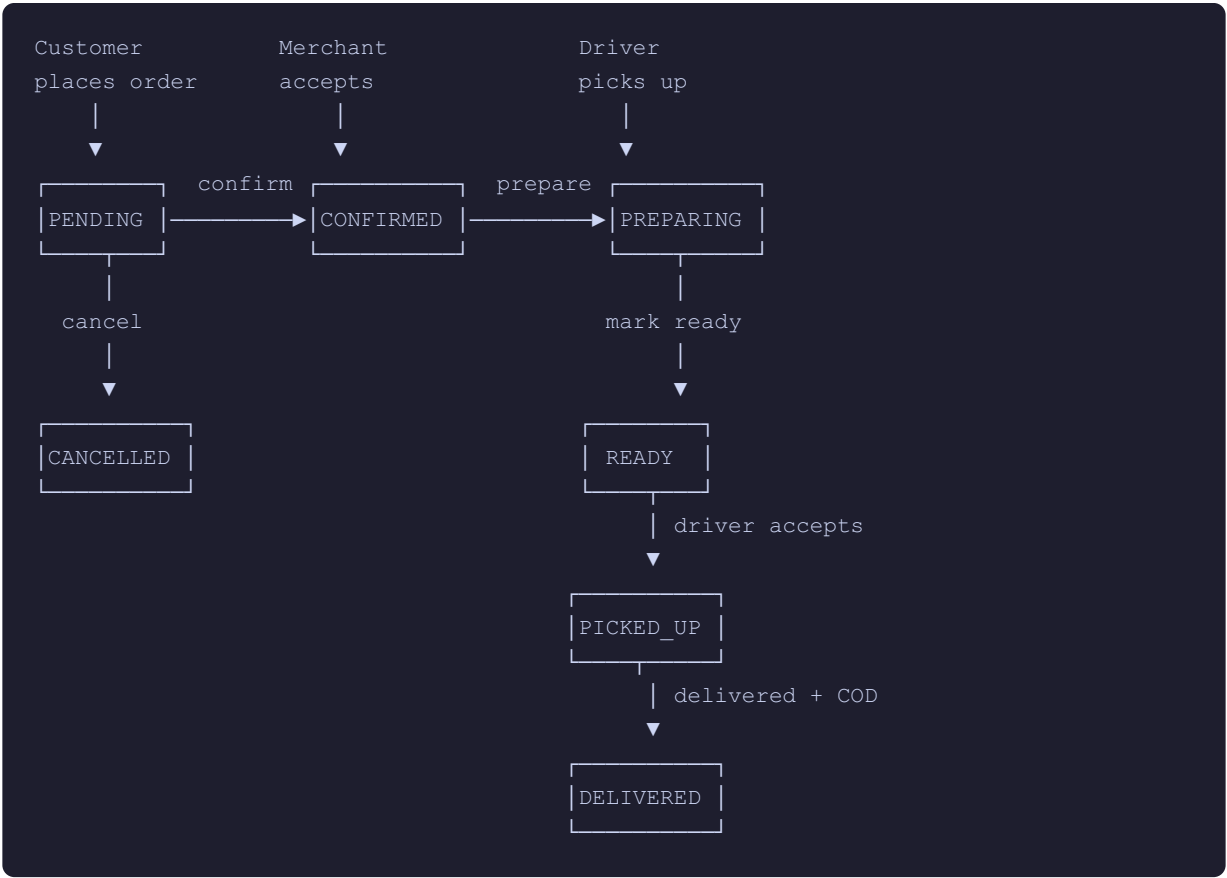
Merchant verification uses fields on Merchant : owner_ktp_url , business_license_url , store_photo_url , verification_status , verification_note

4.3 POS/Kasir Models (inspired by finansial-mac)

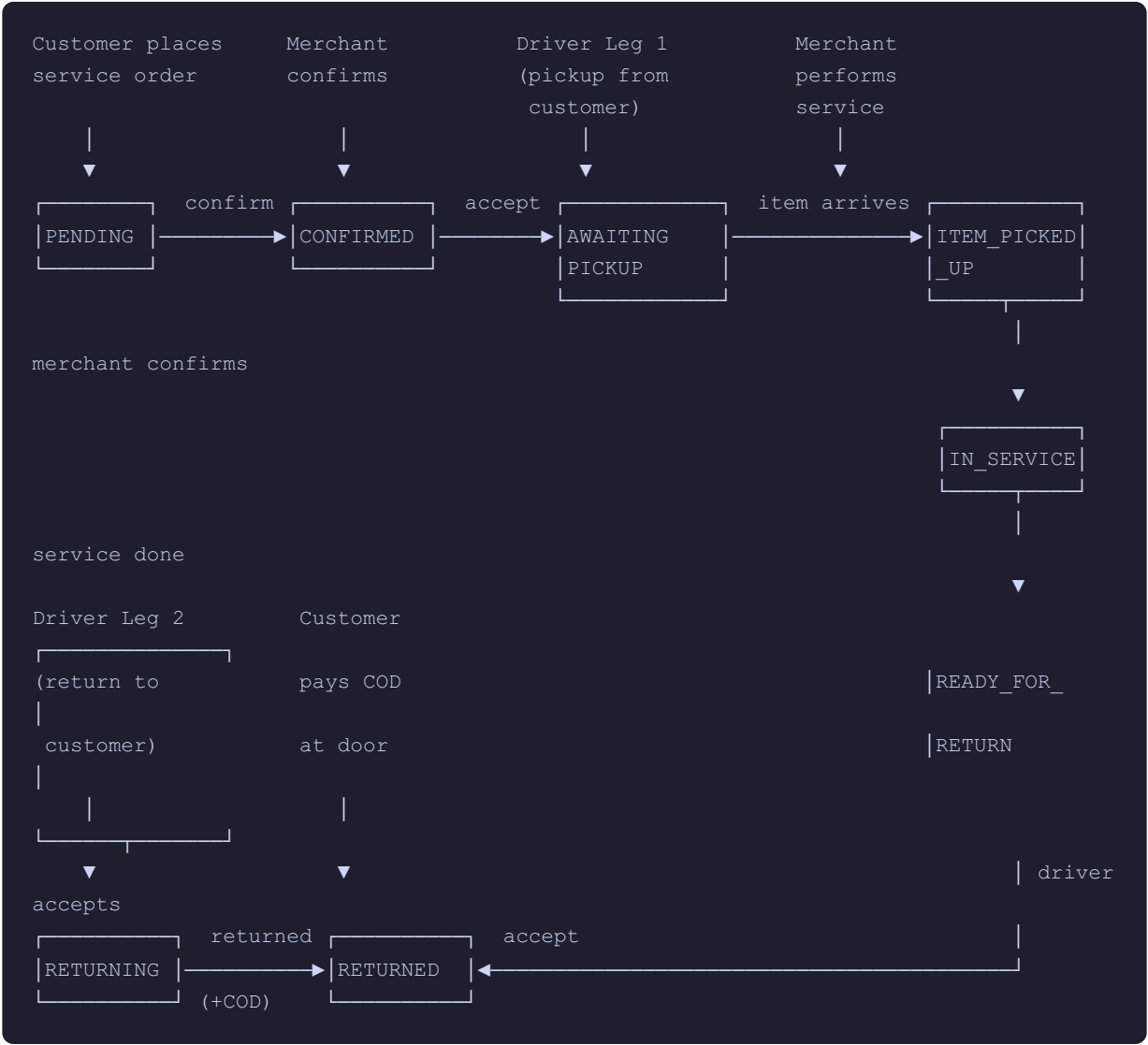
#	Model	finansial-mac Equivalent
16	StockMovement	pergerakan_stok (in/out/opname/void)
17	PosTransaction	jurnal + invoice (header kasir)
18	PosTransactionItem	invoice_item
19	MerchantReceivable	piutang (auto-created on tab payment)
20	MerchantReceivablePayment	bayar_piutang
21	MerchantPayable	hutang supplier
22	MerchantPayablePayment	bayar_hutang

5. Order Status Machines

5.1 Food / Ecommerce Orders



5.2 Service (Jasa) Orders – 2 Driver Legs



6. API Reference (60+ endpoints)

6.1 Authentication

Method	Endpoint	Auth	Description
POST	/auth/register	—	Register user. Customer → active immediately. Driver/Merchant → inactive, pending verification
POST	/auth/login	—	Login, get JWT tokens

6.2 Public — Merchants & Services

Method	Endpoint	Description
GET	/merchants	Food merchants (verified + active)
GET	/merchants/nearby?lat=&lng=&radius=	Nearby food merchants
GET	/merchants/search?q=	Search by keyword
GET	/merchants/:id	Merchant detail
GET	/merchants/:id/products	Full menu with categories
GET	/service-merchants?category=	Service merchants (laundry/bengkel/etc.)
GET	/service-merchants/:id/services	Services offered

6.3 Customer

Method	Endpoint	Description
POST	/orders	Place food order (COD pricing)
GET	/orders	List my food orders
GET	/orders/:id	Food order detail
POST	/orders/:id/cancel	Cancel pending food order
POST	/service-orders	Book service (laundry, bengkel, etc.)
GET	/service-orders	List my service orders
GET	/service-orders/:id	Service order detail
POST	/service-orders/:id/cancel	Cancel pending service order

6.4 Merchant Owner

Method	Endpoint	Description
POST	/my-store	Register merchant (multipart: name, address, docs)
GET	/my-store/status	Verification status check
GET	/my-store	Get store + full product catalog
PUT	/my-store	Update store profile
PUT	/my-store/toggle-open	Toggle open/closed
PUT	/my-store/toggle-self-deliver	Toggle self-delivery mode
PUT	/my-store/self-delivery-fee	Set self-delivery fee
GET	/my-store/my-deliveries	Self-delivery active orders
POST	/my-store/my-deliveries/:id/pickup	Mark self-delivery picked up
POST	/my-store/my-deliveries/:id/deliver	Mark self-delivery delivered
POST/PUT/DELETE	/my-store/categories/*	Category CRUD
POST/PUT/DELETE	/my-store/products/*	Product CRUD
POST/DELETE	/my-store/variants/*	Variant CRUD
GET	/restaurant-orders	Food order queue
POST	/restaurant-orders/:id/accept	Accept food order
POST	/restaurant-orders/:id/preparing	Mark preparing
POST	/restaurant-orders/:id/ready	Mark ready for pickup
GET	/my-service-orders	Service order queue
POST	/my-service-orders/:id/confirm	Confirm service order
POST	/my-service-orders/:id/in-service	Mark in service
POST	/my-service-orders/:id/ready	Mark ready for return

6.5 POS / Kasir (Merchant)

All under `/my-store/pos`

Method	Endpoint	Description
POST	<code>/transactions</code>	Create POS sale (cash/qr/card/tab). Auto-decrements stock. Auto-creates piutang for tab.
GET	<code>/transactions</code>	List with date filter + summary
GET	<code>/transactions/:id</code>	Detail with items
POST	<code>/transactions/:id/void</code>	Void + reverse stock (retur)
GET	<code>/products</code>	Products with stock status + low-stock alerts
PUT	<code>/products/:id/stock</code>	Manual stock adjustment (in/opname)
GET	<code>/products/:id/stock-history</code>	Movement history
GET/ POST	<code>/receivables</code>	List + create piutang
GET	<code>/receivables/:id</code>	Detail + payment history
POST	<code>/receivables/:id/pay</code>	Record payment
GET/ POST	<code>/payables</code>	List + create hutang supplier
GET	<code>/payables/:id</code>	Detail + payment history
POST	<code>/payables/:id/pay</code>	Record payment
GET	<code>/reports/summary</code>	Period summary (revenue, HPP, gross profit, tx count)
GET	<code>/reports/laba-rugi</code>	P&L: Pendapatan → HPP → Laba Kotor → Beban → Laba Bersih
GET	<code>/reports/arus-kas</code>	Cash flow by payment method + piutang/hutang
GET	<code>/reports/stok</code>	Stock status + total stock value

6.6 Driver

Method	Endpoint	Description
POST	/driver/apply	Submit application + document upload (multipart)
GET	/driver/application	Check application status + is_active
GET	/driver-orders/available	Available food orders
POST	/driver-orders/:id/accept	Accept food delivery
POST	/driver-orders/:id/pickup	Mark picked up from merchant
POST	/driver-orders/:id/deliver	Mark delivered + update cash balance
GET	/driver/service-orders/available	Available service jobs (pickup + return)
POST	/driver/service-orders/:id/accept-pickup	Accept leg 1 (jemput dari customer)
POST	/driver/service-orders/:id/item-picked-up	Barang diambil dari customer
POST	/driver/service-orders/:id/accept-return	Accept leg 2 (antar balik ke customer)
POST	/driver/service-orders/:id/returned	Delivered + collect COD

6.7 Admin

Method	Endpoint	Description
GET	/admin/dashboard/stats	Live KPI: orders, merchants, drivers, customers, revenue, driver cash
GET/PUT	/admin/settings/:key	System settings CRUD
GET	/admin/merchants	List all merchants
PUT	/admin/merchants/:id/verify	Approve/reject merchant + set verification_note
GET	/admin/drivers	List all drivers
PUT	/admin/users/:id/toggle-active	Suspend/activate any user
GET	/admin/driver-applications?status=	List driver applications
PUT	/admin/driver-applications/:id/review	Approve/reject driver (auto-creates Driver record)
GET	/admin/drivers/:id/deposits	Driver COD deposit history
POST	/admin/drivers/:id/deposits	Record COD deposit from driver
GET	/admin/customers	List all customers
GET	/admin/orders?status=	All orders
GET	/admin/settlements	Platform-level financial summary
GET	/admin/settlements/merchants	Per-merchant pending payouts + settlement history
POST	/admin/settlements/merchants/:id/process	Create settlement for a period
PUT	/admin/settlements/:id/mark-paid	Mark settlement paid
GET/POST/PUT/DELETE	/admin/promotions/*	Promotions CRUD
PUT	/admin/promotions/:id/toggle	Activate/deactivate promo
GET	/health	Health check

7. Financial Models

7.1 Food / Ecommerce COD

Food Base Price:	IDR	50,000	
+ Platform Markup (15%):	IDR	7,500	
Customer Pays (Food):	IDR	57,500	
+ Delivery Fee (inside zone):	IDR	15,000	
TOTAL CUSTOMER PAYS CASH:	IDR	72,500	
Distribution:			
→ Merchant Receives:	IDR	50,000	
→ Driver Earns:	IDR	11,250	(75% delivery fee)
→ KUWRIR Revenue:	IDR	11,250	(markup + 25% delivery)

7.2 Service (Jasa) COD

Service Base Price (e.g. laundry 4kg × Rp 8.000):	IDR	32,000	
+ Platform Markup (15%):	IDR	4,800	
+ Round-trip Delivery Fee (flat):	IDR	20,000	
TOTAL CUSTOMER PAYS COD (saat barang kembali):	IDR	56,800	
Distribution:			
→ Merchant Receives:	IDR	32,000	(base service price)
→ Driver Earns:	IDR	15,000	(75% delivery fee, covers both legs)
→ KUWRIR Revenue:	IDR	9,800	(markup IDR 4.800 + 25% delivery IDR 5.000)

7.3 POS / Kasir (In-store, no delivery)

Product Base Price:	IDR 50,000 (full goes to merchant – no platform markup)
Payment Methods:	cash gris card tab (credit)
Tab → auto-creates MerchantReceivable (piutang)	
Merchant tracks:	
- Piutang (customer credit/tab)	
- Hutang (supplier payables)	
- Laba Rugi (P&L) per period	
- Arus Kas (cash flow by payment method)	
- Stok (inventory with low-stock alerts)	

7.4 Configurable System Parameters

Key	Default	Description
<code>platform_markup_percentage</code>	15%	Markup on food/service base price
<code>delivery_commission_percentage</code>	25%	KUWRIR's cut of delivery fee
<code>delivery_base_fee_inside_zone</code>	IDR 15,000	Food delivery flat fee
<code>delivery_fee_per_km_outside</code>	IDR 10,000/km	Beyond 5km surcharge
<code>service_delivery_fee_round_trip</code>	IDR 20,000	Service pickup + return flat fee

8. Registration & Verification Flows

8.1 Customer Registration

```
Register (name, phone, email, password, role=customer)
→ is_active = TRUE immediately
→ JWT token returned → can order immediately
```

8.2 Driver Registration

```
Register (role=driver) → is_active = FALSE, no token returned
→ POST /driver/apply (multipart):
  - vehicle_type, vehicle_plate, vehicle_year, vehicle_color, vehicle_brand
  - ktp (foto KTP)
  - sim (foto SIM C/A)
  - stnk (foto STNK)
  - selfie (selfie + KTP)
  - vehicle_photo (foto kendaraan)
→ DriverApplication created (status=pending)
→ Admin reviews at /driver-applications
→ Approve → is_active=TRUE + Driver record created
→ Reject → driver notified, can re-submit
```

8.3 Merchant Registration

```
Register (role=merchant) → is_active = FALSE, no token returned
→ POST /my-store (multipart):
  - name, description, phone, address, latitude, longitude
  - owner_ktp (foto KTP pemilik)
  - business_license (SIUP/IUMK – optional)
  - store_photo (foto toko)
→ Merchant record created (is_verified=false, verification_status=pending)
→ Admin reviews at /merchants (Approve / Reject + note)
→ Approve → is_verified=TRUE, is_active=TRUE
→ Reject → merchant notified, can re-register
```

8.4 Document Storage

Currently: local disk `./uploads/driver-docs/` and `./uploads/merchant-docs/`
Swap to R2: replace body of `upload.Save()` in `backend/internal/upload/upload.go` — no other code changes needed.

9. Merchant Types

Type	type field	service_category	Order Flow
Food / Restaurant	<code>food</code>	—	Standard food order flow
Laundry	<code>service</code>	<code>laundry</code>	Service order flow (pickup → clean → return)
Bengkel / Workshop	<code>service</code>	<code>bengkel</code>	Service order flow (pickup → repair → return)
Cleaning	<code>service</code>	<code>cleaning</code>	Service order flow (technician visits)
Salon	<code>service</code>	<code>salon</code>	Service order flow (home visit)

Service merchants also get access to the **POS/Kasir** feature for walk-in customers.

10. POS/Kasir Financial Integration

Inspired by `finansial-mac` (Python accounting app). Mapping:

finansial-mac	KUWRIR POS/Kasir
pemasukan (per produk)	POST /pos/transactions
piutang + bayar_piutang	MerchantReceivable (auto on tab) + pay endpoint
hutang + bayar_hutang	MerchantPayable + pay endpoint
pergerakan_stok KELUAR	StockMovement type=out (on POS sale)
pergerakan_stok MASUK	StockMovement type=in (on restock)
retur_penjualan	POST /pos/transactions/:id/void
laba_rugi	GET /pos/reports/laba-rugi
arus_kas	GET /pos/reports/arus-kas
hpp_kalkulator	Product.CostPrice + PosTransactionItem.UnitCost

11. Infrastructure

11.1 Docker Compose Services

Service	Image	Port	Purpose
postgres	postgis/postgis:16-3.4	5432	Primary database
redis	redis:7-alpine	6379	Caching
valhalla	ghcr.io/gis-ops/valhalla	8002	Route calculations
nominatim	mediagis/nominatim	8003	Address geocoding

11.2 Production Cost (Single VPS)

Component	Monthly Cost
VPS (4 vCPU, 8GB RAM)	~\$48/month
Cloudflare R2 (image storage)	~\$0.15/month
Domain	~\$1/month
Total	~\$49/month

12. Current Status

Phase	Status	Description
Phase 1	 Complete	Foundation (Backend, Admin Panel, Flutter scaffolds)
Phase 2	 Complete	Restaurant & Menu System
Phase 3	 Complete	Cart, Orders & COD Pricing Engine
Phase 4	 Complete	Driver App + Order Fulfillment
Phase 5	 Complete	Admin Panel Integration (Settlements, Dashboard)
Phase 6a	 Complete	POS/Kasir + Financial Integration (finansial-mac inspired)
Phase 6b	 Complete	Admin Panel — Full feature completion (live KPI, deposit, settlements, promos)
Phase 6c	 Complete	Driver & Merchant Registration + Verification + Document Upload
Phase 6d	 Complete	Service (Jasa) Orders — Laundry, Bengkel, Salon, Cleaning
Phase 7	 Planned	Reviews + Ratings, WebSocket real-time tracking, Push Notifications
Phase 8	 Planned	Image Upload → Cloudflare R2, Digital Payments (QRIS/Xendit)

13. Quick Start

Prerequisites

- Go 1.22+, Node.js 20+, Flutter 3.x, Docker

```
# 1. Start infrastructure
cd backend && docker compose up -d

# 2. Configure environment
cd backend && cp .env.example .env

# 3. Start backend API
cd backend && go run cmd/server/main.go

# 4. Start admin panel
cd admin_panel && npm install && npm run dev

# 5. Start mobile apps
cd customer_app && flutter pub get && flutter run
cd driver_app && flutter pub get && flutter run
cd merchant_app && flutter pub get && flutter run
```

Default Admin Credentials

- **Phone:** 080000000000
- **Password:** admin123